CLEARRO

HARNESS ENGINEERING

AZURE TECH TALK · PRATIK DHANAVE

CLEARROUTE

AGENDA

WHERE WE'RE GOING

1
The idea

Which harness we mean, the thesis behind buying it from managed services, and the two Azure layers that do the work.

2
Patterns & code

Seven production patterns — guardrails to human-in-the-loop — each shown as the real Python that ships in the repo today.

3
The Azure path

How every local stand-in swaps for its managed service — Agent Service, Cosmos DB, AI Search, Content Safety, App Insights.

4
Run it yourself

Project status, the exact uv commands to run the suite and the API locally, and three takeaways to carry home.

A DEMO AGENT MEETS PRODUCTION

WHY THIS TALK

PROMPT INJECTION #1

One crafted message and the agent obeys the attacker instead of you — there is nothing between the user and the model.

CRASH MID-RUN #2

A rate limit at step nine of ten and the whole run restarts from zero — every completed step repeated, every side effect doubled.

UNTRUSTED CODE #3

The model writes code and it executes in your process, on your filesystem, with your credentials. Nothing contains it.

TOTAL AMNESIA #4

Every conversation starts from nothing — no history, no facts, no context the user already gave you three turns ago.

FIRST: WHICH "HARNESS"?

The term is overloaded — two different things share the name. This project is the first one.

THIS PROJECT

Runtime harness

The scaffolding wrapped around a deployed agent — guardrails, memory, sandboxed tools, orchestration, human-in-the-loop. From the Hendrixer/harness-engineering course.

NOT THIS PROJECT

Dev-time harness

OpenAI's "humans steer, agents execute" — the environment and feedback loops that let coding agents write your software. Same words, entirely different layer of the stack.

We're building the harness an agent runs INSIDE, not the harness a team builds software WITH.

THE THESIS: CHANGE THE ALTITUDE

NOT THE SYLLABUS

BUY

Let Azure supply the harness

Prefer managed primitives — Agent Service, Content Safety, Cosmos DB, AI Search — over from-scratch infrastructure you then have to run forever.

OWN

Spend the effort where it counts

Orchestration, guardrails, and observability are the real work. The plumbing underneath them becomes configuration, not code.

LEARN

All seven patterns survive

You still learn every course pattern — but as configuration and orchestration on managed services, not as plumbing you maintain.

RULE

Keep implementations thin

If a lesson can be solved by configuring a managed service, don't hand-roll it. This working rule is written into the repo's guidance.

The original course builds the whole harness by hand in TypeScript. Same syllabus here — different altitude: managed primitives underneath, our judgement on top.

TWO LAYERS DO THE WORK

The division of labour

AZURE PROVIDES

AI Foundry · Agent Service

The agents themselves: model calls plus the tool loop, managed threads for short-term memory, and sandboxed built-in tools — Code Interpreter, File Search, Function, AI Search grounding.

the agents

YOU BUILD

Microsoft Agent Framework

The harness around them: middleware and guardrails, durable checkpointed workflows, multi-agent orchestration, and human-in-the-loop — the unified successor to Semantic Kernel and AutoGen.

the harness around the agents

OUR OWN CODE

FastAPI on Container Apps

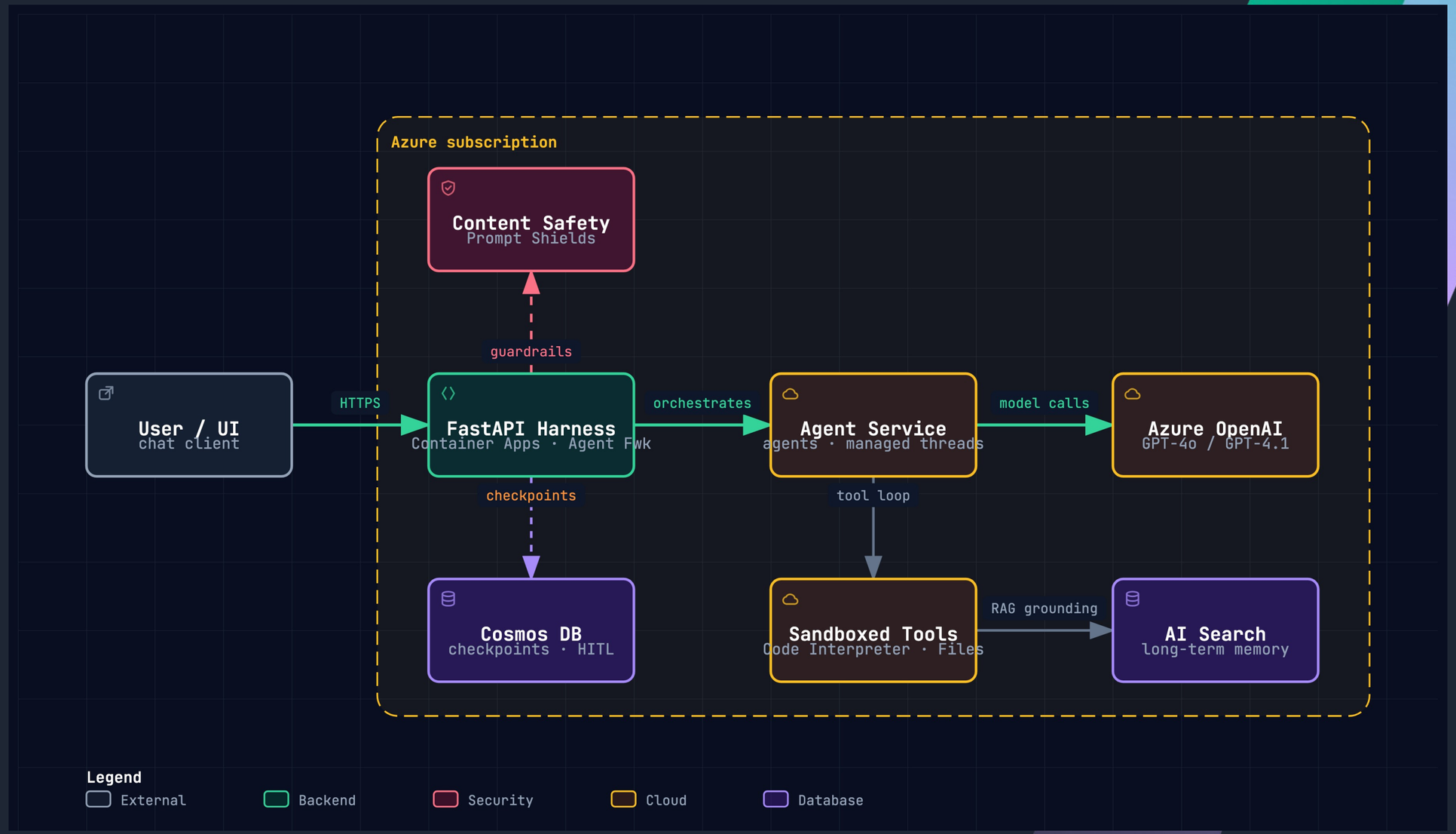
A thin orchestration API that drives the agents. It shrinks to composition — routes, policies, observability — precisely because everything below it is managed by the two layers on the left.

composition only

Agent Framework is public preview — pin exact versions and re-verify APIs before each lesson.

TARGET ARCHITECTURE

One FastAPI layer composes managed services — agents, guardrails, checkpoints, and memory are Azure's job, not ours.



archify · docs/presentation/harness-azure.html (interactive: pan, zoom, focus, export)

THE ONE CONVENTION: THE SEAM PATTERN

Every Azure dependency is stubbed behind a seam, so the whole suite runs with no cloud and no network.

- A typing.Protocol — or an injectable callable / rule table — is the swap point.
- The local implementation's docstring names the Azure service it stands in for.
- It documents honestly where the local stand-in is weaker: "state the leak."

```python
# durable/checkpoint.py — same interface, local SQLite today, Cosmos DB later
class CheckpointStore(Protocol):
    def load(self, run_id: str) -> Checkpoint | None: ...
    def save(self, checkpoint: Checkpoint) -> None: ...
```

Swap local → Azure without touching the workflow logic. Match this pattern when adding code.

THE SEVEN LESSONS

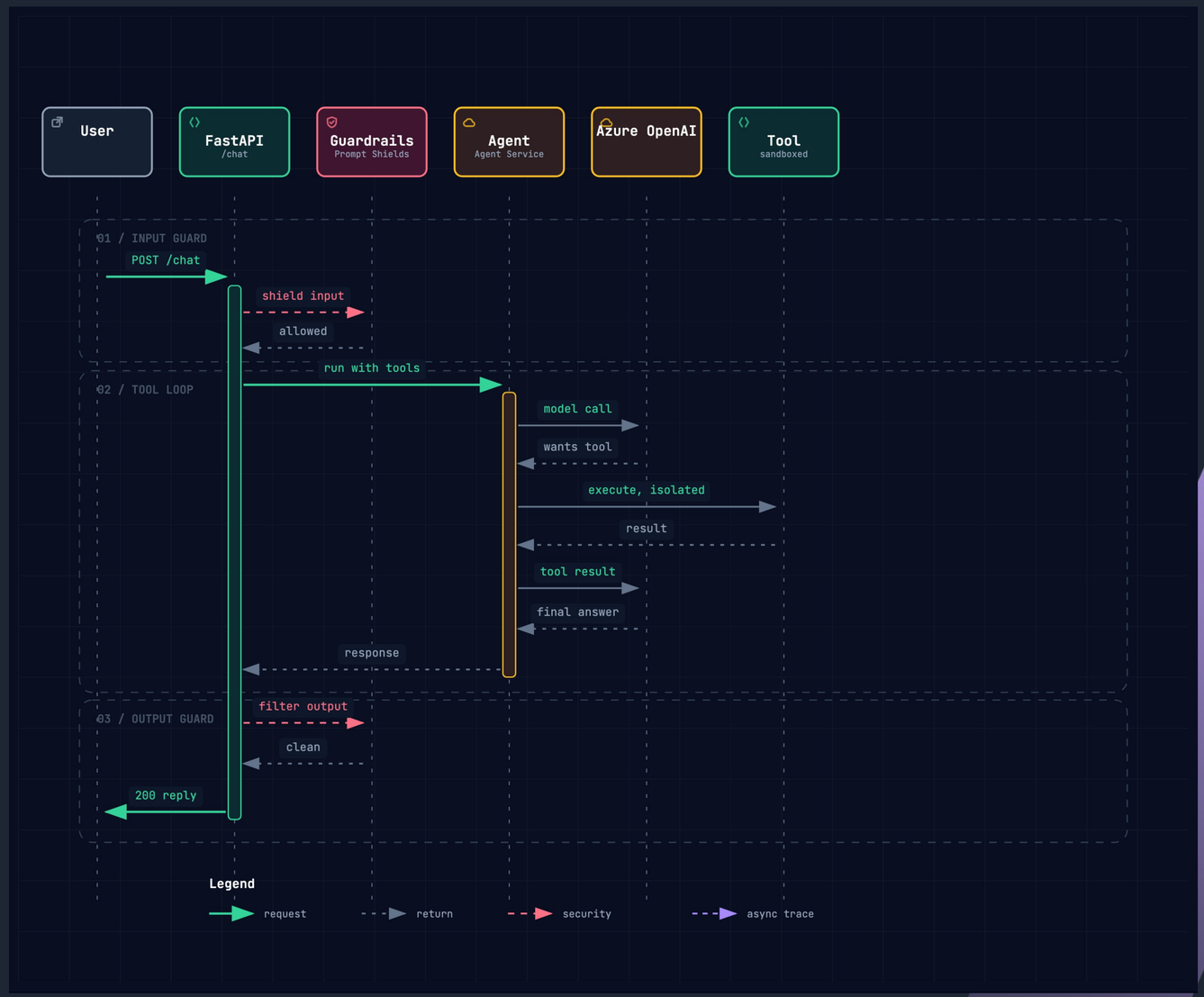
Seven build increments — every one implemented and unit-tested locally today, framed as "Azure provides X / you build Y."

#	PATTERN	YOU BUILD (LOCAL MODULE)	
1	**Agent Harness**	`guardrails.py + app.py`	local ✓
2	**Durable Execution**	`workflow.py + durable/`	local ✓
3	**Secure Sandboxing**	`sandbox.py`	local ✓
4	**Advanced Memory**	`memory/ (thread · knowledge · summarize)`	local ✓
5	**Orchestration**	`orchestration/router.py`	local ✓
6	**Hierarchical Supervision**	`orchestration/supervisor.py`	local ✓
7	**Human-in-the-Loop**	`hitl.py (built on durable/)`	local ✓

"local ✓" = runnable + unit-tested with Azure stubbed behind a seam. The Azure-wired implementations are the next increment.

LESSON 1 · THE GUARDED REQUEST

Guardrails wrap the model on the way in and out; the tool loop runs inside — the Lesson 1 shape, end to end.



archify · docs/presentation/harness-request.html (interactive: pan, zoom, focus, export)

LESSON 1 · GUARDRAILS

Hard-block at the front door: empty, oversize, and injection-marked input never reaches the model — with an obfuscation-folding second pass.

```python
# guardrails.py — hard-block at the front door
def check_input(text: str) -> GuardrailResult:
    if not text or not text.strip():
        return GuardrailResult(allowed=False, reason="empty input")
    if len(text) > MAX_INPUT_CHARS:
        return GuardrailResult(allowed=False, reason="input too long")

    lowered = text.lower()
    for marker in INJECTION_MARKERS:
        if marker in lowered:
            return GuardrailResult(allowed=False, reason="possible prompt injection")

    folded = normalize(text)  # fold homoglyph / zero-width obfuscation
    if folded != lowered and any(m in folded for m in INJECTION_MARKERS):
        return GuardrailResult(allowed=False, reason="obfuscated injection")

    hit = worst(text)  # heuristic attack families (role-play, authority, ...)
    if hit is not None:
        return GuardrailResult(allowed=False, reason=f"{hit.kind}: {hit.reason}")
    return GuardrailResult(allowed=True)
```

src/harness/guardrails.py (abridged)

SAFETY LAYER

A MARKER LIST IS TRIVIALLY BYPASSED

1

normalize

Folds obfuscation — homoglyphs, zero-width characters, spacing tricks — before matching, so a disguised "ignore previous instructions" cannot slip past the substring check.

2

detectors

Heuristic attack-family detectors run over the folded text — indirect injection, authority escalation, role-play jailbreaks — beyond what a flat marker list can express.

3

redteam

A corpus of attack strings plus a scorer, so the layer is measured against itself on every test run rather than assumed to work.

4

state the leak

Still heuristic, not ML — the docstring says so plainly. Managed Prompt Shields remain the production answer. No false confidence.

LESSON 2 · DURABLE EXECUTION

Checkpoint after every step; resume skips completed work. The store is a Protocol — SQLite locally, Cosmos DB in production, same workflow logic.

```python
# durable/checkpoint.py — the seam: SQLite locally, Cosmos DB in production
class CheckpointStore(Protocol):
    def load(self, run_id: str) -> Checkpoint | None: ...
    def save(self, checkpoint: Checkpoint) -> None: ...

# workflow.py — checkpoint after every step; resume skips completed work
def run_durably(run_id: str, steps: list[Step], store: CheckpointStore) -> dict:
    cp = store.load(run_id)
    start, state = 0, {}
    if cp is not None:
        start = cp.step + 1  # resume after the last completed step
        state = dict(cp.state)

    for index in range(start, len(steps)):
        _name, fn = steps[index]
        update = fn(state)
        if update:
            state.update(update)
        store.save(Checkpoint(run_id=run_id, step=index, state=state))
    return state
```

src/harness/durable/checkpoint.py · src/harness/workflow.py (abridged)

LESSON 3 · SECURE SANDBOXING

Untrusted code runs in an isolated interpreter (-I) with a hard timeout — a runaway loop is killed, not awaited. A teaching boundary, not a real one.

```
# sandbox.py — untrusted code in an isolated subprocess with a hard timeout
def run_code(code: str, timeout: float = DEFAULT_TIMEOUT_SECONDS) -> SandboxResult:
    try:
        proc = subprocess.run(
            [sys.executable, "-I", "-c", code],  # -I: isolated interpreter
            capture_output=True,
            text=True,
            timeout=timeout,
        )
    except subprocess.TimeoutExpired as exc:
        return SandboxResult(ok=False, stdout=_as_text(exc.stdout),
                             stderr="timed out", exit_code=None, timed_out=True)

    return SandboxResult(ok=proc.returncode == 0, stdout=proc.stdout,
                         stderr=proc.stderr, exit_code=proc.returncode,
                         timed_out=False)
```

src/harness/sandbox.py (abridged)

LESSON 4 · ADVANCED MEMORY

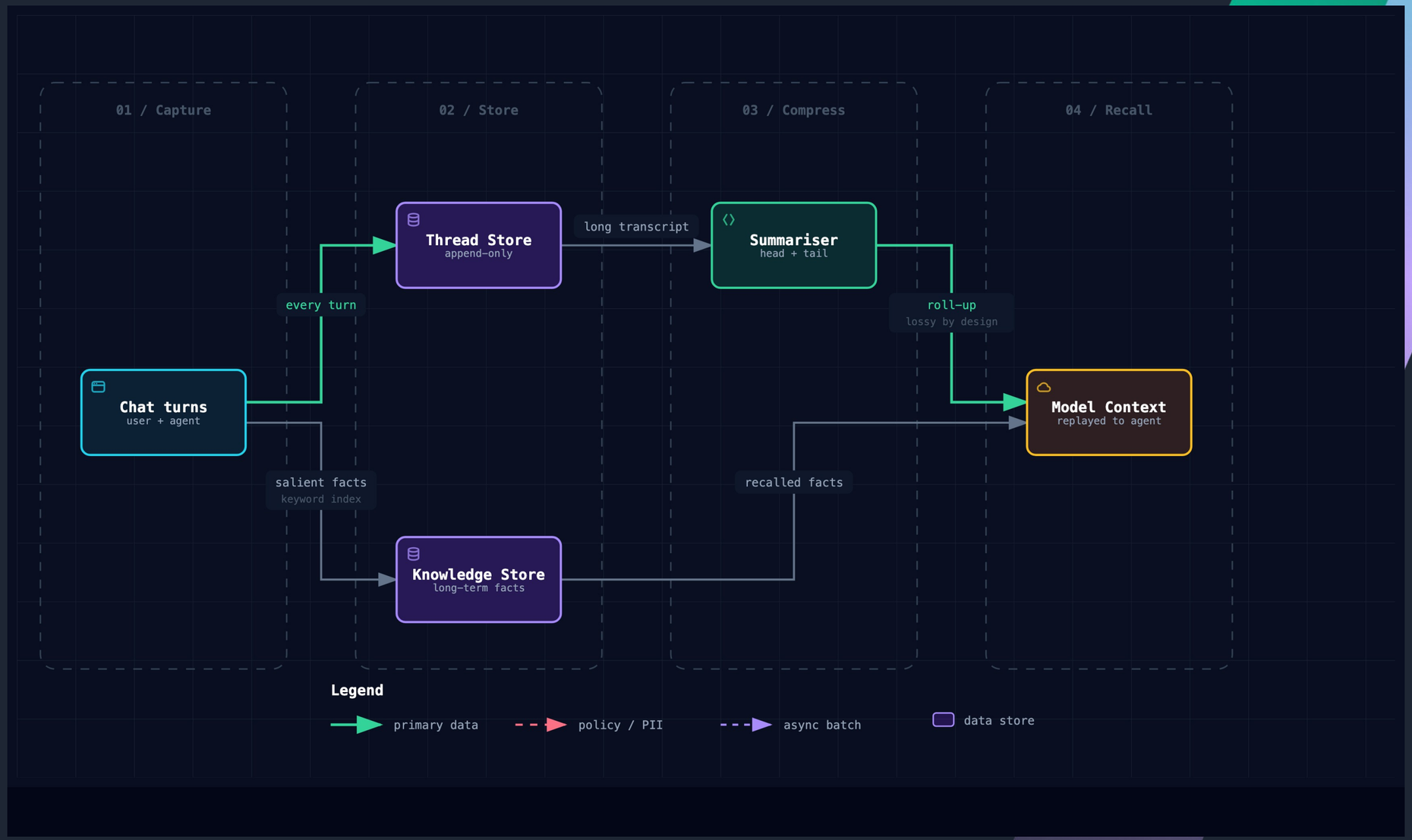
Summarisation is lossy by design — keep the first and last turns, drop the middle, and leave the raw thread recoverable in the thread store.

```
# memory/summarize.py — lossy roll-up; the raw thread stays recoverable
def summarize_thread(messages, max_chars=500, keep=2) -> str:
    keep = max(1, keep)
    lines = [f"{m.role}: {m.content}" for m in messages]
    full = "\n".join(lines)
    if len(full) <= max_chars:
        return full  # fits — nothing is lost
    if len(messages) <= 2 * keep:
        return full  # no droppable middle: a marker would hide nothing
    head, tail = lines[:keep], lines[-keep:]
    omitted = len(messages) - 2 * keep
    marker = f"[...{omitted} earlier turns omitted...]"
    return "\n".join([*head, marker, *tail])
```

src/harness/memory/summarize.py (abridged)

LESSON 4 · MEMORY FLOW

Every turn is kept verbatim; the roll-up is lossy on purpose — a bad summary is always recoverable from the thread store.



archify · docs/presentation/harness-memory.html (interactive: pan, zoom, focus, export)

LESSON 5 · ROUTING & HANDOFFS

Triage maps intent to a specialist. Locally a first-match keyword table; the leak — no notion of meaning — is stated in the docstring.

```python
# orchestration/router.py — deterministic triage: keyword -> specialist
DEFAULT_RULES = {
    "invoice": Specialist.BILLING, "refund": Specialist.BILLING,
    "bug": Specialist.CODE,        "stack trace": Specialist.CODE,
    "paper": Specialist.RESEARCH,  "cite": Specialist.RESEARCH,  # ...
}

class Router:
    def route(self, request: str) -> RouteDecision:
        """First keyword (in table order) found in the request wins."""
        lowered = request.lower()
        for keyword, specialist in self.rules.items():
            if keyword in lowered:
                return RouteDecision(specialist=specialist.value,
                                     matched_intent=keyword)
        return RouteDecision(specialist=Specialist.FALLBACK.value,
                             matched_intent=None)
```

src/harness/orchestration/router.py (abridged)

LESSON 6 · SUPERVISION

Fan out, isolate faults, keep order: a raising worker is contained in its own SubResult, and results return in decomposition order, not finish order.

```
# orchestration/supervisor.py — fan out, isolate faults, keep order
def run(self, task: str) -> Aggregate:
    subtasks = self.decompose(task)
    results = [None] * len(subtasks)  # slot by index, not completion order

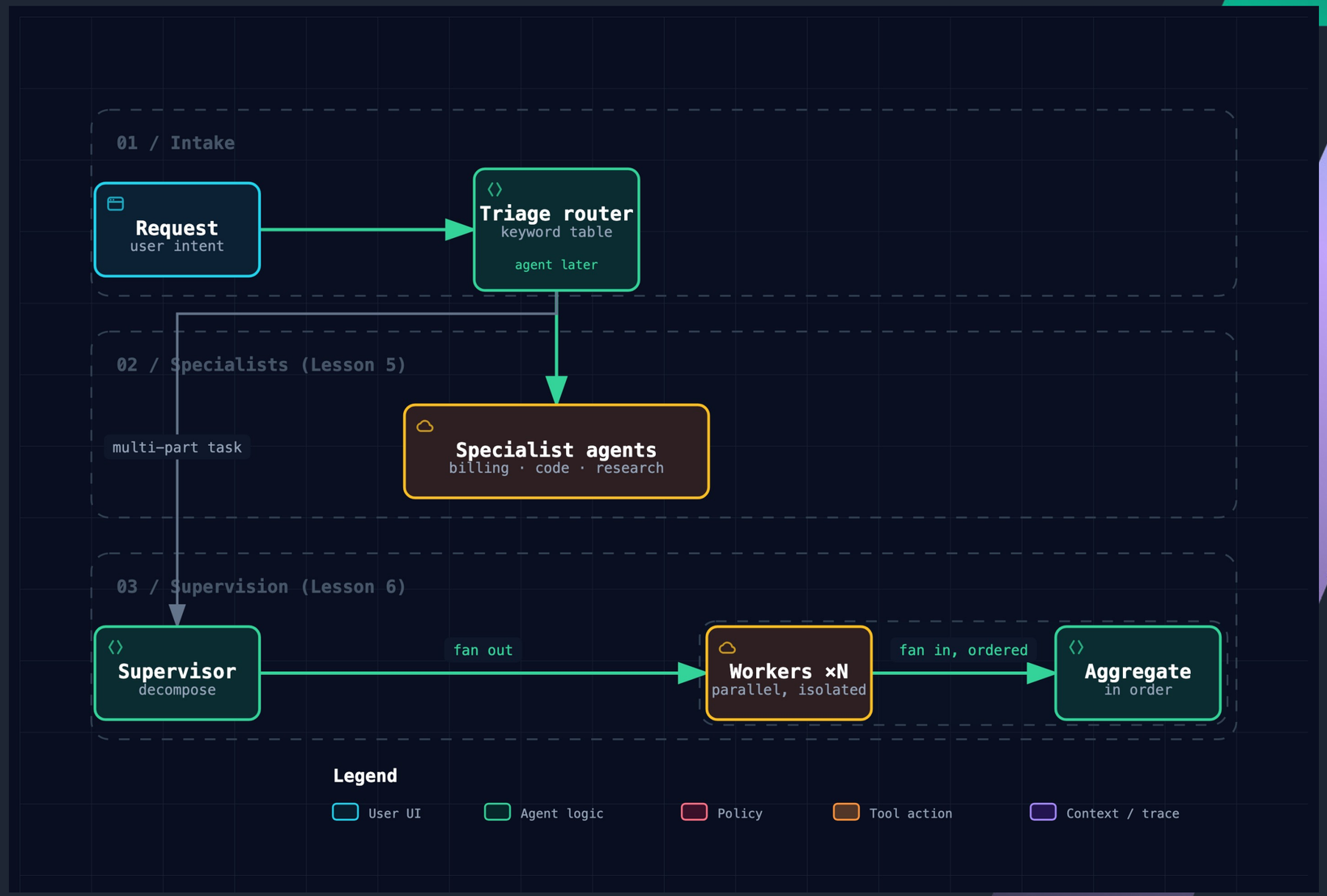
    with ThreadPoolExecutor(max_workers=self.max_workers) as pool:
        futures = {pool.submit(self.worker, sub): i
                   for i, sub in enumerate(subtasks)}
        for future, i in futures.items():
            try:
                results[i] = SubResult(subtasks[i], future.result(), error=None)
            except Exception as exc:  # one bad sub-agent cannot sink the batch
                results[i] = SubResult(subtasks[i], output=None, error=str(exc))

    succeeded = sum(1 for r in results if r.error is None)
    return Aggregate(results, succeeded, failed=len(results) - succeeded)
```

src/harness/orchestration/supervisor.py (abridged)

LESSONS 5–6 · ORCHESTRATION

Triage hands off to one specialist; a supervisor splits real work across parallel workers and fans results back in, in order.



archify · docs/presentation/harness-orchestration.html (interactive: pan, zoom, focus, export)

LESSON 7 · HUMAN-IN-THE-LOOP

The pause IS a durable checkpoint — built on Lesson 2's store. Approval, rejection, and deadline resolve against injected time; a late yes is void.

```python
# hitl.py — the pause IS a checkpoint; durability comes free from Lesson 2
def request_approval(run_id, prompt, store, now, *, timeout_seconds=3600.0):
    store.save(Checkpoint(run_id=run_id, step=0, state={
        "status": "awaiting_approval", "prompt": prompt,
        "requested_at": now, "timeout_seconds": timeout_seconds,
    }))
    return ApprovalPending(run_id=run_id, prompt=prompt)

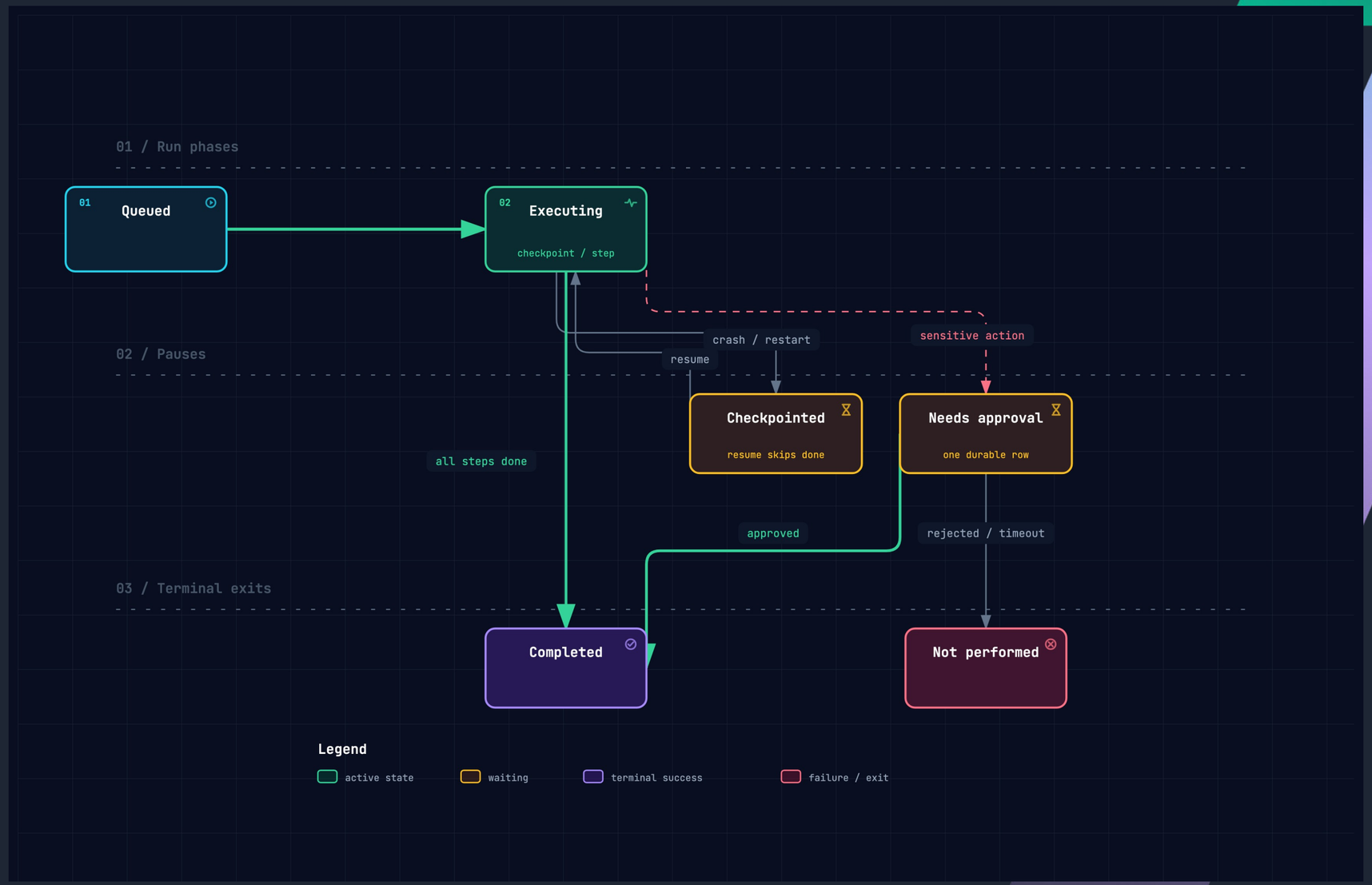
def resume(run_id, decision, store, now, *, action=None):
    cp = _load_pending(run_id, store)
    if now - cp.state["requested_at"] > cp.state["timeout_seconds"]:
        return ApprovalOutcome("timed_out", ...)  # a late "yes" is void
    if not decision:
        return ApprovalOutcome("rejected", ...)   # action NOT performed

    update = action(dict(cp.state)) if action is not None else {}
    _resolve(cp, store, now, status="approved", performed=True, extra=update)
    return ApprovalOutcome("approved", "human approved; action performed")
```

src/harness/hitl.py (abridged)

LESSONS 2 & 7 · RUN LIFECYCLE

Crashes resume from checkpoints; sensitive actions park for approval — both are just durable rows, and a late yes is void.



archify · docs/presentation/harness-run-lifecycle.html (interactive: pan, zoom, focus, export)

DOCS ARE PART OF THE TEST SUITE

Knowledge lives in the repo and is mechanically validated — stale docs turn the test suite red.

- tests/test_docs.py checks every .md and .excalidraw: balanced code fences, resolvable anchors, valid Excalidraw JSON, no HTML-escape artifacts.
- The heavier Mermaid render check is opt-in — HARNESS_RENDER_MERMAID=1, needs the mmdc CLI — so the default suite stays fast and cloud-free.
- Two glossaries cross-reference every concept, including a full LangGraph → this repo → Azure mapping (docs/langgraph-glossary.md).

A small echo of "repository as system of record": if an agent — or a teammate — can't see it in-context, it doesn't exist.

STATUS & HOW TO RUN IT

All seven lessons are implemented locally and tested; the Azure wiring is the next increment — it swaps seams, not logic.

- 7 / 7 lessons implemented — 101 deterministic tests, no cloud, no network, no keys.
- Durable and HITL tests write real checkpoints to a local .harness/ SQLite store.
- Despite the Go-style path, this is Python — uv provisions ≥3.11 and everything else.

```
# run the full suite (no cloud needed)
uv run --extra dev pytest

# one test
uv run --extra dev pytest tests/test_guardrails.py::test_normal_input_allowed

# the API locally
uv run uvicorn harness.app:app --reload
curl localhost:8000/health
```

TAKEAWAYS

01 **Thin over managed.** The harness is mostly configuration on Azure primitives — Agent Service, Content Safety, Cosmos DB — not from-scratch infrastructure.

02 **The seam pattern + "state the leak".** Build and test everything offline, then swap local for Azure without touching the workflow logic — and never hide where a stand-in is weaker.

03 **The seven patterns are the vocabulary.** Production agents need to guard, persist, sandbox, remember, route, supervise, and ask a human. Everything else is composition.

Questions?

`docs/architecture.md` · `docs/azure-implementation-plan.md` · `docs/glossary.md` · `docs/langgraph-glossary.md`

Pratik Dhanave · pratik.dhanave@clearroute.io